

Versiones de Java

Nomenclatura:

Desde las primeras versiones de Java hasta la versión 8, los nombres utilizados eran 1.X (por ejemplo 1.8). A partir de la versión 9, el sistema de nombres cambió y se eliminó el 1.

Versión 8: <https://www.oracle.com/java/technologies/javase/8-whats-new.html>

Añadidas las Expresiones Lambda, que permiten tratar a las funciones como el argumento de un método o el código como datos.

El paquete **java.util.stream** añadido al API de Collections, permite operaciones masivas sobre colecciones, integrándolas con las Expresiones Lambda.

Añadida nueva API para gestión de fechas y horas, Date-Time API. Se añade el paquete **java.time**.

Modificados los paquetes java.lang y java.util:

Parallel Array Sorting
Standard Encoding and Decoding Base64
Unsigned Arithmetic Support

Versión 9: <https://docs.oracle.com/javase/9/whatsnew/toc.htm>

Se marca como deprecated el plug-in de Java y el API de Applets.

Se permiten métodos privados en las interfaces (private static).

Nuevos métodos para construir colecciones:

```
List<String> lista = List.of("uno", "dos", "tres");  
Set<String> conjunto = Set.of("uno", "dos");  
Map<String, String> mapa = Map.of("clave1", "dato1", "clave2", "dato2");
```

Nuevos métodos para Stream.

Añadido JShell como herramienta para ejecutar comandos inmediatos desde consola.

Versión 10: <https://www.oracle.com/java/technologies/javase/10-relnote-issues.html>

Modificado el API de colecciones:

Añadidos métodos para crear copias de colecciones (**List.copyOf**, **Set.copyOf**, **Map.copyOf**).

Añadidos métodos a Stream para crear colecciones inmutables.

Añadida la palabra reservada **var** para declarar variables.

```
var lista = new ArrayList<String>(); //El tipo se infiere a partir del constructor
```

Versión 11: <https://www.oracle.com/java/technologies/javase/11-relnote-issues.html>

JavaFX ya no está incluido en el JDK. Ahora es un elemento descargable por separado desde <https://openjfx.io/>

Añadido soporte para Unicode 10

Versión final del cliente **HttpClient**

La palabra reservada **var** se puede utilizar para declarar los parámetros de las Expresiones Lambda

Se pueden ejecutar los ficheros .java directamente contra la máquina virtual

Versión 12: <https://www.oracle.com/java/technologies/javase/12-relnote-issues.html>

Añadido soporte para Unicode 11

Añadido soporte para formatear número de forma compacta en java.text.NumberFormat con el método NumberFormat.getCompactNumberInstance

1000 se puede formatear como 1K

Versión 13: <https://www.oracle.com/java/technologies/javase/13-relnote-issues.html>

Añadido soporte para Unicode 12.1

Versión 14: <https://www.oracle.com/java/technologies/javase/14-relnote-issues.html>

Versión definitiva de Expresiones Switch:

```
boolean isTodayHoliday;
switch (day) {
    case "MONDAY":
    case "TUESDAY":
    case "WEDNESDAY":
    case "THURSDAY":
    case "FRIDAY":
        isTodayHoliday = false;
        break;
    case "SATURDAY":
    case "SUNDAY":
        isTodayHoliday = true;
        break;
    default:
        throw new IllegalArgumentException("What's a " + day);
}
```

```
boolean isTodayHoliday = switch (day) {
    case "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY" -> yield
false;
    case "SATURDAY", "SUNDAY" -> yield true;
    default -> throw new IllegalArgumentException("What's a " + day);
};
```

Versión 15: <https://www.oracle.com/java/technologies/javase/15-relnote-issues.html>

Añadido soporte para Unicode 13

Añadida versión definitiva de bloques de texto o cadenas de texto de varias líneas:

```
String texto = """
    Lorem ipsum dolor sit amet, consectetur adipiscing \
    elit, sed do eiusmod tempor incididunt ut labore \
    et dolore magna aliqua.\
    """;
```

Versión 16: <https://www.oracle.com/java/technologies/javase/16-relnote-issues.html>

Añadida versión definitiva de Record:

Un registro (record) permite obviar la implementación de clases inmutables con constructor, getters, equals, hashCode y toString

```
public class Persona {
    private final String nombre;
    private final int edad;
    public Persona (String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
    // más getter, equals, toString y hashCode
}
```

```
public record Persona (String nombre, int edad) { }
```

Añadida versión definitiva de Pattern Matching para instanceof:

```
Object obj = "PRUEBA";
if (obj instanceof String) {
    String t = (String) obj;
    // resto del código...
}
```

```
Object obj = "TEST";
if (obj instanceof String t) {
    // resto del código...
}
```

Versión 17: <https://www.oracle.com/java/technologies/javase/17-relnote-issues.html>

Añadida versión definitiva de las clases selladas (sealed):

Permiten limitar la herencia, indicando que clases pueden ser hijas de esta clase

```
public sealed class Persona permits Alumno, Profesor {  
...  
}
```

```
public final class Alumno extends Persona {  
...  
}  
public non-sealed class Profesor extends Persona {  
...  
}
```

Obligatoriamente las clases hijas deben ser declaradas final, non-sealed o sealed

Versión 18: <https://www.oracle.com/java/technologies/javase/18-relnote-issues.html>

Establecido UTF-8 como codificación por defecto para las APIs de Java SE

Añadido un servidor web como herramienta de línea de comandos (jwebserver)

Versión 19: <https://www.oracle.com/java/technologies/javase/19-relnote-issues.html>

Añadido soporte para Unicode 14

Añadidos nuevos métodos al API de colecciones para la creación de HashMap y HashSet:

```
HashMap.newHashMap  
LinkedHashMap.newLinkedHashMap  
WeakHashMap.newWeakHashMap  
HashSet.newHashSet  
LinkedHashSet.newLinkedHashSet
```

Versión 20: <https://www.oracle.com/java/technologies/javase/20-relnote-issues.html>

Añadido soporte para Unicode 15

Eliminados los métodos deprecated Thread.suspend(), Thread.resume() y Thread.stop()

Aumentadas los patrones de comparación del switch, que ahora incluyen:

- **Tipos de datos:**

```
static String formatterPatternSwitch(Object obj) {  
    return switch (obj) {  
        case Integer i -> String.format("int %d", i);  
        case Long l    -> String.format("long %d", l);  
        case Double d  -> String.format("double %f", d);  
        case String s  -> String.format("String %s", s);  
        default        -> obj.toString();  
    };  
}
```

- **Null:**

```
static void testFooBarNew(String s) {  
    switch (s) {  
        case null          -> System.out.println("Oops");  
        case "Foo", "Bar" -> System.out.println("Great");  
        default            -> System.out.println("Ok");  
    }  
}
```

- **Se añade la cláusula WHEN para especificar subcasos:**

```
static void testStringEnhanced(String response) {  
    switch (response) {  
        case null -> { }  
        case "y", "Y" -> {  
            System.out.println("You got it");  
        }  
        case "n", "N" -> {  
            System.out.println("Shame");  
        }  
        case String s  
        when s.equalsIgnoreCase("YES") -> {  
            System.out.println("You got it");  
        }  
        case String s  
        when s.equalsIgnoreCase("NO") -> {  
            System.out.println("Shame");  
        }  
        case String s -> {  
            System.out.println("Sorry?");  
        }  
    }  
}
```

Versión 22: <https://www.oracle.com/java/technologies/javase/22-relnote-issues.html>

Añadidas variables y patrones sin nombre. Se representan con un guión bajo _

```
static int count(Iterable<Order> orders) {  
    int total = 0;  
    for (Order _ : orders)    // Unnamed variable  
        total++;  
    return total;  
}
```

```
switch (ball) {  
    case RedBall _    -> process(ball);           // Unnamed pattern variable  
    case BlueBall _   -> process(ball);           // Unnamed pattern variable  
    case GreenBall _  -> stopProcessing();         // Unnamed pattern variable  
}
```

Añadido soporte para Unicode 15.1

Añadida clase ListFormat para procesar listas de acuerdo a la especificación del locale

Versión 23: <https://www.oracle.com/java/technologies/javase/23-relnote-issues.html>

Abandonado el desarrollo de String templates

Añadidos métodos a la clase Console, que toman un Locale como argumento

```
public Console format(Locale locale, String format, Object ... args)  
public Console printf(Locale locale, String format, Object ... args)  
public String readLine(Locale locale, String format, Object ... args)  
public char[] readPassword(Locale locale, String format, Object ... args)
```

Versión 24: <https://www.oracle.com/java/technologies/javase/24-relnote-issues.html>

Añadido soporte para Unicode 16